Day 13 (Morning Session)

React Routing, Forms and Event Handling

Complete beginner-friendly lecture notes with code, expected output, interview questions and revision material

Item	Details
Instructor	Abhishek Purohit
Lecture date	2 November 2026
Subject	React.js front end (JavaScript, Vite project on port 5173)
Topics covered	Routing concept; react-router-dom (BrowserRouter, Routes, Route); Link vs anchor tag; programmatic routing with useNavigate; forms in React; single state object for form data; event object and event.target; spread operator; generic handleChange with the name attribute and computed keys; onSubmit and preventDefault; end-of-class Q and A (payments, PDF generation, debouncing and throttling)

How to read these notes

Text with no label is the instructor's lecture content, rewritten in clear English (spoken Hindi/English mix converted to English, filler removed).

Green boxes titled 'Supplementary' are extra explanations added by me to help understanding. They were NOT said by the instructor. Red boxes highlight common mistakes. Yellow boxes are exam or interview tips.

Contents

1. Project Setup and Cleanup 4
2. What is Routing? 5
3. Implementing Routing: react-router-dom 6
4. Building the Navigation Bar and Pages 7
5. Connecting Everything: URL to Component Mapping 9
6. Problem with the Anchor Tag and the Link Component 12
7. Programmatic Routing with useNavigate 14
8. Forms in React: Building the Registration Screen 17
9. How to Deal with Form Data: The Three Steps 18
10. Handling the onChange Event 20

11. The Event Object and event.target 21

12. Updating the State Correctly: The Spread Operator 23

13. One Generic handleChange Using the name Attribute 25

14. Submitting the Form: onSubmit and preventDefault 27

15. End-of-Class Questions and Answers 30

16. Revision Material 32

Topic coverage checklist (checked against the full transcript)

#	Topic discussed in the lecture	Section
1	Project setup: npm i, cleaning App.jsx and main.jsx, npm run dev on 5173	1
2	What routing is; real-world examples (ChatGPT, Notion, Gmail, YouTube); SPA idea	2
3	React has no built-in routing; library vs framework; react-router-dom	3
4	Folder structure (components, pages); NavigationBar, Home, AboutUs, ContactUs	4
5	Role of anchor tags vs role of React Router; mapping URL to component	5
6	Route (path, element), Routes, BrowserRouter; path must match the link URL	5
7	Problem: anchor tag reloads the browser; Link component and the `to` prop	6
8	Programmatic routing; UserDetails page; useNavigate hook and navigate()	7
9	Forms in React; RegistrationForm UI; /register route and link	8
10	Three steps of form handling; single state object vs many states	9
11	onChange (and onKeyUp); handler functions; console log proof	10
12	Event object passed automatically by React; synthetic event; event.target.value	11
13	Bug: setFormData overwrites the state; fix with spread operator	12
14	One generic handleChange; name attribute; computed property key	13
15	onSubmit, handleSubmit, page reload problem, preventDefault, JSON form data	14
16	Q and A: payment integration, PDF/Excel generation, debouncing and throttling	15
17	Revision: summary, syntax, differences, mistakes, interview Q and A, exercises, checklist	16

1. Project Setup and Cleanup

The lecture starts a new project named **routing demo** using React with JavaScript. The instructor installs dependencies manually instead of using a generator again.

1.1 Steps performed

1. Open the project folder in VS Code (File > Open Folder > select the `routing demo` folder).
2. Open the terminal and run `npm i`. This installs every dependency already listed in `package.json`.
3. Clean `App.jsx`: remove all the default Vite code, including the `useState` import, and keep only one empty `div` inside the component.
4. In `main.jsx`, remove the line `import './index.css'` (we do not need the default styling).
5. Start the development server with `npm run dev`. It launches on port 5173 (`http://localhost:5173`). At this point the page is blank because `App` has no content.

App.jsx after cleanup

```
// src/App.jsx  (cleaned)
function App() {
  return (
    <div>
    </div>
  )
}

export default App
```

main.jsx after cleanup

```
// src/main.jsx  (index.css import removed)
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Explanation

- `npm i` reads `package.json` and downloads the packages into the `node_modules` folder.
- `npm run dev` runs the development script (Vite), which serves the app and reloads when files change.
- `App` is the root component: whatever it returns is what the browser shows.

> **Supplementary: what main.jsx does**
>
> `createRoot(...).render(<App />)` takes the `div` with id `root` from `index.html` and draws the `App` component inside it. `StrictMode` is a development helper that highlights potential problems; it does not change what the user sees.

2. What is Routing?

2.1 The problem: many screens in one application

A real application never has just one screen. A typical web application has a dashboard, a login page, a registration page, a profile page, a settings page, a contact us page and so on. You cannot show every screen inside one component, one below the other (dashboard first, then About Us below it, then Profile below that). The question is: **how do we render and display these different screens?** The answer is routing.

2.2 Definition

> **Definition (instructor)**
>
> **Routing** simply means opening a component on a specific route (a specific URL). Every screen has a corresponding route (URL). When the user clicks a menu item, we change the URL in the browser, and for that URL we open the corresponding component.

This is how a **Single Page Application (SPA)** gives the feeling of having many pages, while in reality every page is just a component.

2.3 Real-world examples shown by the instructor

The instructor used a chat application and Notion as examples. When he moved between menu options such as *New Chat* or *Library*, the URL changed each time, and a different component opened for each URL. Moving to *Welcome to Notion* changed the URL again and opened another component. Every application such as Gmail, Google Drive or YouTube works with this routing mechanism because each has many screens, and each screen is reached through its own URL.

2.4 Two ways a user reaches a route

- The user types the URL in the browser address bar.
- The application gives the user a **navigation bar** (Home, About Us, Contact Us, Services). The user clicks an option, the URL changes and the suitable component opens.

> **Goal of this lecture**
>
> Create a navigation bar with Home, About Us and Contact Us, and open the suitable component for each URL. For example: base URL `/` opens Home, `/about` opens About Us, `/contact` opens Contact Us.

3. Implementing Routing: react-router-dom

3.1 React has no built-in routing

React.js has no inbuilt mechanism to deal with routing. React only gives a small library to create single page applications: state, props, lifecycle and hooks. If you want routing, you must take help of a third-party package. The instructor's analogy: if something is not available at home, you bring it from outside.

Concept	Explanation (from lecture)	Key point
Library	A small set of tools. React gives you state, props, lifecycle and hooks only.	You add other things (routing, HTTP, etc.) yourself.
Framework	A complete package with many built-in features.	Routing and similar needs are included.

3.2 Installing the package

Terminal (stop the server first with Ctrl + C)

```
npm i react-router-dom
```

The instructor stopped the server, ran `npm install react-router-dom`, and waited for the installation. This package supplies predefined components that together build the whole routing mechanism.

3.3 The three core components

Component	Purpose	Notes
`BrowserRouter`	Wraps the entire application to tell React that routing is enabled in the browser.	Final step of the setup. Everything that uses routing must be inside it.
`Routes`	A container (parent) that holds all the `Route` components as its children.	Mandatory: routes must be wrapped inside it.
`Route`	Defines the mapping: for which URL which component must open.	Takes two props: `path` and `element`.

> **Supplementary: version note**
>
> These notes use the `react-router-dom` package as in the lecture (the version-6 style API). Newer major versions publish the same components from the `react-router` package too, but importing from `react-router-dom` continues to work in this course setup.

4. Building the Navigation Bar and Pages

4.1 Folder structure

The instructor chose a managed approach: all separate screens go in one folder and all reusable components go in another. With this the application stays organised.

Folder layout

```
src/
  components/
    NavigationBar.jsx      <- reusable component used on every screen
  pages/                   <- actual screens (could also be named 'screens')
    Home.jsx
    AboutUs.jsx
    ContactUs.jsx
  App.jsx
  main.jsx
```

> **Instructor's note**
>
> `components` holds reusable components used by different screens. `pages` (or `screens`) holds the actual screens such as Home, About Us, Dashboard and so on.

4.2 NavigationBar (first version with anchor tags)

The navigation bar simply shows a few anchor tags. For now the `href` is `#`; later it is changed to real URLs.

NavigationBar.jsx - first version

```
// src/components/NavigationBar.jsx
function NavigationBar() {
  return (
    <div>
      <a href="#">Home</a> |{' '}
      <a href="#">About Us</a> |{' '}
      <a href="#">Contact Us</a>
    </div>
  )
}

export default NavigationBar
```

4.3 The three pages

Home.jsx

```
// src/pages/Home.jsx
export default function Home() {
  return (
    <div>
      <h3>Welcome to Home Page</h3>
      <p>This is some random description for the home page.</p>
    </div>
  )
}
```

AboutUs.jsx (mission and vision added to make it look different from Home)

```
// src/pages/AboutUs.jsx
export default function AboutUs() {
  return (
    <div>
      <h2>About Us</h2>
      <h4>Our Mission</h4>
```

```
      <p>Description of our mission goes here.</p>
      <h4>Our Vision</h4>
      <p>Description of our vision goes here.</p>
    </div>
  )
}
```

ContactUs.jsx (any random phone and email values)

```
// src/pages/ContactUs.jsx
export default function ContactUs() {
  return (
    <div>
      <h2>Contact Us</h2>
      <p>Phone: 9876543210</p>
      <p>Email: admin@gmail.com</p>
    </div>
  )
}
```

> **Supplementary: default export**
>
> In the lecture the instructor wrote 'export function ...' and imported with the file path. In these notes every component uses `export default`, so it is imported as `import Home from './pages/Home'`. Both styles are valid as long as the import matches the export.

5. Connecting Everything: URL to Component Mapping

5.1 Why not render the navbar inside every page?

The navigation bar must appear on every page. One way would be to render `NavigationBar` inside Home, About Us and Contact Us separately. But if the application has fifty or hundreds of screens, writing the navbar inside each one is not ideal. Instead, the navbar is rendered **once inside App.jsx**.

Below the navbar, which component appears cannot be hard coded (you cannot write Home, About or Contact permanently). That decision depends on the **current URL**, and that is the job of routing.

5.2 The mapping decided in the lecture

Browser URL	Component to open	File
`/` (base URL, nothing after the slash)	Home	pages/Home.jsx
`/about`	About Us	pages/AboutUs.jsx
`/contact`	Contact Us	pages/ContactUs.jsx

5.3 Division of responsibility (very important idea)

Who	Job	Does NOT do
Navigation bar (anchor / Link)	Only sends the user to a URL, i.e. changes the URL in the browser.	It does not decide which component opens.
React Router DOM	Looks at the current URL and opens the matching component.	It does not create the navigation links.

So whenever the user hits Home, send the user to the base URL `/`. For About Us send to `/about`. For Contact Us send to `/contact`.

5.4 Phase 1: anchor tags with real URLs (only the URL changes)

First the instructor replaced `#` with real URLs in the anchors and checked only whether the browser URL changes. No component switching works yet.

```
// src/components/NavigationBar.jsx - Phase 1
function NavigationBar() {
  return (
    <div>
      <a href="/">Home</a> |{' '}
      <a href="/about">About Us</a> |{' '}
      <a href="/contact">Contact Us</a>
    </div>
  )
}

export default NavigationBar
```

Expected output: The navbar appears. At first the URL is the base URL (nothing after the slash). Clicking About Us changes the URL to `/about`; clicking Contact Us changes it to `/contact`. The anchor tag has done its job. However, the page body still does not change, because nobody has defined which component belongs to which URL.

5.5 Phase 2: define the mapping with Route

The `Route` component defines the mapping: *for which particular route which component has to be opened*. It takes two props:

- `path` - the URL. It must be written exactly the same as the URL used in the anchor/link.
- `element` - the component that should open for that URL.

> **Rule from the lecture: path and href must be the same**
>
> The link and the route work in correlation. The URL the anchor sends the user to is exactly the URL you must handle in `path`. You cannot send the user to `/about` and handle `/abouts` in the route: it will not listen.

Route for each page

Three Route components (mapping only; not complete yet)

```
<Route path="/" element={<Home />} />
<Route path="/about" element={<AboutUs />} />
<Route path="/contact" element={<ContactUs />} />
```

5.6 Phase 3: wrap the routes in Routes, and the app in BrowserRouter

Defining `Route` alone is not enough. Two compulsory things remain:

1. All `Route` components must be wrapped as children inside a single `Routes` component. `Routes` acts as the container (parent) of all routes. Without it, routing will not work properly.
2. The entire application must be wrapped inside `BrowserRouter` to denote that routing is enabled for the browser. Because the routed components and the navbar are all rendered by `App`, wrapping the content of `App` in `BrowserRouter` covers the whole application.

> **Structure to remember**
>
> BrowserRouter contains Routes. Routes contains Route. Route is where the mapping (path and element) lives.

App.jsx - routing setup

```
// src/App.jsx - complete routing setup
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import NavigationBar from './components/NavigationBar'
import Home from './pages/Home'
import AboutUs from './pages/AboutUs'
import ContactUs from './pages/ContactUs'

function App() {
  return (
    <BrowserRouter>
      <NavigationBar />

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<AboutUs />} />
        <Route path="/contact" element={<ContactUs />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

Step-by-step explanation

1. Import `BrowserRouter`, `Routes` and `Route` from `react-router-dom`, and import the navbar and the three pages.
2. `<BrowserRouter>` is the outermost wrapper; it switches routing on for the whole app.
3. `<NavigationBar />` is placed before `<Routes>`, so it stays visible on every URL.
4. `<Routes>` checks the current browser URL against its child `Route` entries.
5. The `Route` whose `path` matches the URL renders its `element` (the component) at that position, below the navbar.

Execution flow

User opens the app on the base URL `/` -> `Routes` finds `path="/"` -> `Home` renders under the navbar. User clicks About Us -> URL becomes `/about` -> `Routes` finds `path="/about"` -> `AboutUs` renders. Same for `/contact`.

Expected output: On the base URL the Home page shows automatically (the instructor stressed that this is not hard coded: it is the routing mechanism). Click About Us: the About Us component appears. Click Contact Us: the Contact Us component appears.

Common mistakes at this stage

1) Forgetting to wrap `Route` elements inside `Routes`.
2) Not wrapping the app inside `BrowserRouter`.
3) Writing a `path` that does not exactly match the URL used in the link (for example `/about` vs `/aboutus`).
4) Forgetting to import the page components in `App.jsx`.

Supplementary: why the navbar must be inside BrowserRouter

The `Link` component (next section) and hooks such as `useNavigate` only work inside a router. Since `NavigationBar` is rendered inside `<BrowserRouter>` in `App`, it can safely use them. Many projects place `<BrowserRouter>` in `main.jsx` instead; both work as long as everything using routing is inside it.

6. Problem with the Anchor Tag and the Link Component

6.1 Problem: the browser reloads

After the routing setup works, the instructor asked the class to notice something that should not happen: when moving between routes, the **browser reloads**. In a single page application, the browser must not reload when the user navigates between screens or pages.

> **Who is the culprit?**
> Not React Router DOM. The **anchor tag** is the culprit. The default behaviour of an anchor tag is to always request a new page, which makes the browser reload. The anchor tag knows nothing about keeping the page consistent. The routing mechanism itself is working totally fine.

6.2 The solution: Link component

We need something that does the same job as an anchor tag (send the user to a URL) but without reloading the browser. React Router DOM provides this: the **Link** component. It is not the HTML link tag. Its name starts with a capital L, and it is imported from `react-router-dom`.

Feature	Anchor tag `<a>`	`Link` component
Comes from	HTML	react-router-dom
Prop for the URL	`href`	`to`
Browser reload on click	Yes (default behaviour)	No
Fits single page application	No	Yes
Job	Send the user to a URL	Same job, plus no reload

6.3 Improved approach: replace anchors with Link

Import `Link`, replace each anchor tag, and replace `href` with the `to` prop. The instructor mentioned that the value may be given inside curly braces or directly as a string; it depends on you.

NavigationBar.jsx - final version for routing

```
// src/components/NavigationBar.jsx - using Link
import { Link } from 'react-router-dom'

function NavigationBar() {
  return (
    <div>
      <Link to="/">Home</Link> |{' '}
      <Link to="/about">About Us</Link> |{' '}
      <Link to="/contact">Contact Us</Link>
    </div>
  )
}

export default NavigationBar
```

Expected output: Initially the base URL shows Home. Clicking About Us shows the About Us component for `/about` with **no browser reload**. Clicking Contact Us shows Contact Us for `/contact`, again with no reload. This is the proper way to implement routing.

Common mistakes

1) Writing `href` on `Link`: Link does not support it; use `to`.

2) Forgetting to import `Link` from `react-router-dom`.

3) Writing `link` in lowercase: the component name is `Link` with a capital L.

Interview point

Q: Why use Link instead of `<a>` in React? A: `<a>` makes the browser reload and request a new page; `Link` changes the URL without a reload, so the single page application stays intact.

7. Programmatic Routing with useNavigate

7.1 Why do we need it?

So far the user reaches a route by clicking a link. But sometimes the user must be sent to a URL **by writing code**. This is called **programmatic routing** (or programmatic redirection).

> **Use case from the lecture**
> At login time the user is on the login component. When the login succeeds, would you show a link saying 'Login successful, click here to go to the dashboard'? No. As soon as login succeeds, the user should automatically be sent to the dashboard. For that you write code that redirects to a different route.

Concept	Explanation	Example
Declarative routing (link based)	The user clicks a `Link`; the URL changes through default behaviour.	Navbar links
Programmatic routing	Your code decides when to send the user to a URL; no default behaviour exists.	Button click, login success, inside `useEffect`

An anchor tag moves the user to a new URL automatically by default; a normal button does not. So when redirecting on a button click (or any logic), we must send the user ourselves through code.

7.2 Phase 1: create the target page (UserDetails)

UserDetails.jsx - a random table (do not worry about the UI; real UI will be built with MUI later)

```
// src/pages/UserDetails.jsx
export default function UserDetails() {
  return (
    <div>
      <h2>User Details</h2>
      <p>See the user details below.</p>

      <table border="1">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>1</td>
            <td>Suresh</td>
          </tr>
        </tbody>
      </table>
    </div>
  )
}
```

7.3 Phase 2: register the route

Even though the user will not reach this page through the navbar, the route must still be registered. If the application has hundreds of routes, you must register all hundred `Route` components; there is no shortcut. The only thing that differs is *how the user reaches* each route: the first three routes through the navbar, this last one through a button click (programmatic).

Add this inside <Routes> in App.jsx (and import UserDetails)

```
<Route path="/user-details" element={<UserDetails />} />
```

7.4 Phase 3: button and handler in Home

A normal button is placed in the Home component, just after the `h3`. It is not a link: it is an ordinary button whose `onClick` executes custom logic, and that logic sends the user to the desired route.

The useNavigate hook

The function that does the redirect is named `navigate`. But `navigate` cannot be imported directly. To get it, you call a hook named **useNavigate** (imported from `react-router-dom`), and this hook **returns the navigate function**.

> **Definition (as read out from the documentation in class)**
> `useNavigate` returns a function that lets you navigate programmatically in the browser, in response to user interactions or effects. So user actions or events can send the user to a new URL through this function.

The variable name can be anything (`redirectUser`, `performRouting`...), but the commonly used name is `navigate`. Inside `navigate(...)` you pass the URL where the user should be redirected.

Home.jsx - final version

```
// src/pages/Home.jsx - with programmatic routing
import { useNavigate } from 'react-router-dom'

export default function Home() {
  const navigate = useNavigate()

  const viewUserDetailsHandler = () => {
    // any logic can run here, then redirect
    navigate('/user-details')
  }

  return (
    <div>
      <h3>Welcome to Home Page</h3>
      <button onClick={viewUserDetailsHandler}>View User Details</button>
      <p>This is some random description for the home page.</p>
    </div>
  )
}
```

Step-by-step explanation

1. `useNavigate()` is called at the top of the component. It returns the `navigate` function, stored in a constant.
2. `viewUserDetailsHandler` is defined to run when the button is clicked. The function is passed as a reference (no parentheses) so React calls it only when the click happens.
3. Inside the handler, `navigate('/user-details')` changes the browser URL to `/user-details` without a reload.
4. `Routes` sees the new URL and opens `UserDetails`.

Expected output: On Home, clicking *View User Details* opens the User Details page at `/user-details`. Clicking About Us shows About Us, then Contact Us, then Home; clicking the button again sends the user back to User Details. The instructor confirmed that the real-time application (later) will use these same techniques for navigation.

Where else can navigate be used?

It can be used on a button click, on a different user action, or even inside a `useEffect` where, while running logic, you decide that the user must go to a certain URL.

Common mistakes

1) Trying `import { navigate } from 'react-router-dom'`: navigate is not a direct import; call the `useNavigate` hook.

2) Calling the hook inside the click handler: hooks must be called at the top level of the component, not inside nested functions (supplementary rule of hooks).

3) Forgetting to register the route in `App.jsx`: navigate will change the URL but no component will open.

4) Writing `onClick={viewUserDetailsHandler()}`: this calls the function immediately while rendering.

Supplementary: Link vs navigate

Use `Link` when the user chooses to click a visible link. Use `navigate` when your code decides the redirect (after login, after a successful save, after a timer, and so on).

8. Forms in React: Building the Registration Screen

Forms are another aspect needed in every real-time application. We must learn how to read data from a form, how to handle the form, and how to submit the form data. The same thing was done earlier in core HTML with JavaScript and DOM manipulation (reading form data and then submitting it). Now the same concept is done in a React application.

8.1 Create the page, route and link

1. Create a new screen `pages/RegistrationForm.jsx`.
2. Add a link for it in the navigation bar: `<Link to="/register">Register</Link>`.
3. Register the route in `App.jsx`: `<Route path="/register" element={<RegistrationForm />} />` (and import the component).

Expected output: After restarting (`npm run dev`), a new Register link appears in the navbar. Clicking it opens `/register` and shows the RegistrationForm component.

8.2 The form UI

The form has four input fields (name, phone number, email, password) and a submit button. `br` tags are put between fields only to create extra gaps. If required, more fields can be added.

RegistrationForm.jsx - static form

```
// src/pages/RegistrationForm.jsx - UI only (first version)
export default function RegistrationForm() {
  return (
    <div>
      <h3>Register Today</h3>

      <form>
        <div>
          <label>Name</label>
          <input type="text" placeholder="Enter name" />
        </div>
        <br />
        <div>
          <label>Phone</label>
          <input type="text" placeholder="Enter phone" />
        </div>
        <br />
        <div>
          <label>Email</label>
          <input type="email" placeholder="Enter email" />
        </div>
        <br />
        <div>
          <label>Password</label>
          <input type="password" placeholder="Enter password" />
        </div>
        <br />
        <button type="submit">Sign Up</button>
      </form>
    </div>
  )
}
```

> **Supplementary: JSX reminder**
>
> In JSX every tag must be closed. The instructor noted an `input` can be closed in the opening tag itself (`<input ... />`). Also `type="email"` and `type="password"` change how the browser treats the field.

9. How to Deal with Form Data: The Three Steps

The question: how do we read the data of a form in a React component (class or function component, both work the same way)? The instructor explained that there is a fixed number of steps.

Step	What we do	When it happens
1	Create a state to contain (represent) the form data.	When the component is defined.
2	As soon as the user types in a field, capture the value and put it in that state.	On every change in an input field (onChange event).
3	At the time of submit, read the form data from the **state** and submit it.	On click of the submit button (onSubmit event).

9.1 Step 1: create a state for the form data

The component state represents the form data given by the user. Whatever the user types must be held inside the component state. In a function component we use `useState`, which gives two things: the state reference and a function to update it (`formData` and `setFormData`).

One state per field, or one state for the whole form?

	Option 1: separate state for each input	Option 2: one state object (preferred)
Idea	`name/setName`, `phone/setPhone`, `email/setEmail`, ...	One JSON object with a key per field.
Four fields	Four different states	One state
Maintenance	A large amount of state is a hectic job to maintain.	Entire form data is under one container.
Complexity	More state means more complexity.	Simple.
Re-rendering	More chances of unnecessary re-rendering.	Fewer unnecessary re-renders.
Verdict	Works, but not ideal.	More ideal and more practical. Always go with this if one state is enough.

The state is initialised with an object containing a key for each field. Because there is no user data at the start, every value is an empty string.

Step 1 - state for the form data

```
import { useState } from 'react'

export default function RegistrationForm() {
  // Step 1: one state object that represents the entire form
  const [formData, setFormData] = useState({
    name: '',
    phoneNumber: '',
    email: '',
    password: '',
  })

  // ... return (UI) ...
}
```

> **Supplementary: key naming**
>
> The lecture said 'phone number' for the key and the matching name attribute. Camel case (`phoneNumber`) is used in these notes. Whatever you choose, the state key and the input's `name` attribute must be spelled the same (see Section 13).

9.2 Step 2: capture the value as the user types

The state exists only to hold the data, so it must be filled. When should the value be taken? Not on the submit button. The moment is **as soon as the user types something into the field**. When the user types in the name field, take the value and assign it to the name key. When the user types in the phone field, take the value at that same moment and assign it to the phone key. In this way the entire state keeps getting populated.

Event to use: the instructor recalled `onKeyUp` from core HTML; to be precise, you can use `onKeyUp` or, generally, the **onChange** event.

9.3 Step 3: read from the state at submit time

> **The key discussion (important)**
> When the data has to be submitted, whom do you ask for it: the component state or the input fields? The answer is the **component state**. If you asked the input fields again at submit time, then why did we create the state at all? Since all fields have already been copied into the state while the user was typing, at submission you only deal with the state data. If the state is empty, tell the user their fields have no data; if it has data, that is your form data and it can be submitted.

The class had some audio trouble during this explanation, so the instructor repeated it. Summary of the repeated explanation: the state was created to hold what the user enters; step 2 captures input at the time the user is typing (not at submit time); so at submission the entire form data is already inside the state, and you ask the component for the state data rather than interacting with the input fields.

Question	Answer
When is input captured into the state?	While the user is typing (onChange / onKeyUp).
From where is data read at submit time?	From the component state, not from the input fields.
Does submit button click capture the input?	No. By then the state already has all the data.

10. Handling the onChange Event

To implement step 2 we write logic that runs when the user starts typing. The event `onChange` is fired as soon as the field value changes: typing a new character or erasing one fires the event.

10.1 Proof that the event fires (first handler)

Handler for the name field

```
// inside RegistrationForm (before return)
const handleNameChange = () => {
  console.log('typing')
}

// in the JSX
<input type="text" placeholder="Enter name" onChange={handleNameChange} />
```

Expected output: Reload the page, open the browser console and type in the Name field. The word `typing` appears in the console once for every character typed (or erased). This is the proof that we can handle the exact moment when the user types.

10.2 What should the handler do?

1. Read the field value that the user has typed.
2. Put that value in the state.

Recall the core JavaScript way: give the field an id, get its object with `document.getElementById`, then use `.value`. In React we also need the **object of the input field** (for the password field we need the password field's object), but the way of getting it is different.

> **Do not do this in React**
>
> You can still give an id and use `getElementById` in React, but you should not. The instructor's sayings: 'if you have been given a gold pot, do not go out begging' and 'if you have a swimming pool, do not bathe with a bucket'. Use React's own internal mechanism, explained next.

11. The Event Object and event.target

11.1 Internal mechanism (explained step by step)

When you handle an event such as `onClick` or `onChange`, do you pass the function's **reference**, or do you **call** the function? You pass the reference. Therefore you are not calling the function manually; **React calls it behind the scenes**.

> **The secret rule**
> Whenever a function is called on an event (onClick, onChange, onKeyUp, onSubmit, onKeyDown ...), React automatically passes one argument to that function. You do not see it being passed, but it happens every time. That argument **represents the object of the event that occurred**.

Event that occurred	Argument becomes
onChange	Object of the onChange event
onClick	Object of the onClick event
onSubmit	Object of the onSubmit event
onKeyDown	Object of the onKeyDown event

11.2 Seeing the event object

In the Home component the instructor temporarily took an argument in `viewUserDetailsHandler` (called `obj`), commented out `navigate`, and logged it.

Logging the event object (temporary experiment)

```
const viewUserDetailsHandler = (obj) => {
  console.log(obj)
  // navigate('/user-details')    <- commented out for this demo
}
```

Expected output: Clicking the button prints a **SyntheticBaseEvent** in the console. This unusual looking key-value structure is the object of the occurred (click) event. Expanding it shows many key and value pairs. (Remember to uncomment `navigate` afterwards.)

> **Supplementary: what is a SyntheticEvent?**
> React wraps the browser's native event in its own object (a SyntheticEvent) so that events behave the same in all browsers. It offers the familiar properties such as `target` and methods such as `preventDefault()`.

11.3 Applying it to handleNameChange

Taking the event argument

```
const handleNameChange = (event) => {
  console.log('typing', event)
}
```

Expected output: Go to /register and type `Q`. The console shows the SyntheticBaseEvent; its type is `change`, i.e. the object of the onChange event. React passed it automatically.

11.4 event.target

The event object has many key-value pairs. One property is **target**. The instructor asked the class what target contains. The class guessed the field data; the instructor corrected this: it contains the **input field's object** (the element on which the event fired), not just the data.

> **Key equivalence**
> `event.target` gives the same thing as `document.getElementById('someId')` gave in core JavaScript: the object of the input element. In core JS we created that object ourselves through the id. In React we get an automated mechanism: when the user touches the name field, the event object tells us on which HTML element the event fired. That is `target`.

Core HTML and JavaScript	React	Result
`document.getElementById('x')`	`event.target`	Object of the input field
`document.getElementById('x').value`	`event.target.value`	Text typed by the user

event.target and event.target.value

```
const handleNameChange = (event) => {
  console.log(event.target)         // the whole <input> element
  console.log(event.target.value)   // what the user typed so far
}
```

Expected output: Typing `Q` logs the whole input element (and `Q`). Typing one more letter (for example `Z`) logs `QZ`, because `value` is the full current content of the field, not only the last character.

12. Updating the State Correctly: The Spread Operator

12.1 Initial (incorrect) approach

Now step 2 of the handler: put the value in the state. The instructor wrote the following and asked the class whether it is correct or has a bug.

Incorrect: overwrites the whole state object

```
const handleNameChange = (event) => {
  setFormData({
    name: event.target.value,
  })
}
```

12.2 Problem

A student correctly answered that phone number, email and password also have to be contained. Explanation: `setFormData` **replaces the older entire object with the new object**; it does not just add keys. The old state contained all four key-value pairs. After this call the new state contains only the `name` key. If the user typed `Rahul`, the state becomes `{ name: 'Rahul' }` and the other three keys are lost.

	State before	State after `setFormData({ name: 'Rahul' })`
Value	`{ name: '', phoneNumber: '', email: '', password: '' }`	`{ name: 'Rahul' }` (others lost)

12.3 Solution: spread operator

Assign the new value to the `name` key but keep the other key-value pairs as they are. Use the spread operator (triple dot). Spread the existing `formData` first so all existing pairs stay, and then write the new value for the `name` key; the later key overrides the earlier value.

Correct: spread the old state and override one key

```
const handleNameChange = (event) => {
  setFormData({
    ...formData,                // keep phoneNumber, email, password as they are
    name: event.target.value,   // update only the name key
  })
}
```

Expected output (state): If the old state was `{ name: '', phoneNumber: '9876', email: '', password: '' }` and the user types `Aman` in name, the new state is `{ name: 'Aman', phoneNumber: '9876', email: '', password: '' }`. Only name changed; the rest remained consistent.

Common mistake

Writing `setFormData({ name: value })` without `...formData`. The other fields get erased every time one field changes.

Supplementary: safer form of the update

When the new state depends on the old state, React also allows `setFormData(prev => ({ ...prev, name: value }))`. This was not shown in the lecture; the lecture's version with `...formData` is fine for this lesson.

12.4 The next problem: repeated functions

The same must be done for phone number (`handlePhoneChange`), email (`handleEmailChange`) and password (`handlePasswordChange`): each handler spreads the form data and assigns a value to its own key. If the form had fifty input fields, would you write fifty functions? Obviously not, because the logic in all of them is exactly the same. Every handler calls `setFormData` and gets its value from `event.target.value`. Only one thing differs: **the key**.

Not a practical approach

```
// Repetition: only the key differs
const handleNameChange  = (e) => setFormData({ ...formData, name: e.target.value })
const handlePhoneChange = (e) => setFormData({ ...formData, phoneNumber: e.target.value })
const handleEmailChange = (e) => setFormData({ ...formData, email: e.target.value })
const handlePasswordChange = (e) =>
  setFormData({ ...formData, password: e.target.value })
```

13. One Generic handleChange Using the name Attribute

13.1 Step towards a single function

Define one function, `handleChange`, and attach it to the `onChange` of **every** input field. Whether the user types in name, phone, email or password, that single function runs. The definition is the same for all; only the key differs.

A single function - but the key is still hard coded

```
const handleChange = (event) => {
  setFormData({
    ...formData,
    name: event.target.value,     // PROBLEM: key is hard coded
  })
}
```

13.2 The new problem

`event.target.value` is fine: for the name field it gives the name value; for the phone field it gives the phone value. The problem is the **key**. Because one function serves all fields, every time the data would be assigned to the `name` key (even if the user typed in the password field, the password text would be stored under `name`). Hard coding any other key (for example phone number) has the same problem.

> **Think approach-wise, not code-wise**
>
> If the user types in the name field the key should become `name`; if in phone, the key should become `phoneNumber`; if in email, `email`; if in password, `password`. The key cannot be hard coded: it must change dynamically based on the field in which the user is typing. So how do we recognise which field is being used?

13.3 Solution: unique identifier through the name attribute

Give every input field a unique identifier. One identifier is `id`, but the instructor said id is never appreciated for forms (it can be used, there is no real problem, but we follow semantics). Instead give each input a **name** property (like the radio buttons earlier, where a common name allowed single selection). The value of each `name` attribute is set to the corresponding **key of the form data**.

Input field	name attribute value	State key it updates
Name field	`name="name"`	`name`
Phone field	`name="phoneNumber"`	`phoneNumber`
Email field	`name="email"`	`email`
Password field	`name="password"`	`password`

Now read the `name` property of the field on which the event occurred: `event.target.name`. Its value becomes the key, and `event.target.value` is the data.

13.4 Computed property names (square brackets)

We need the *value* of a variable to act as a key. From the JSON basics: put square brackets around the expression and the value inside is used as the key. So we write `[event.target.name]`.

Computed key demo (supplementary illustration of the JSON basics mentioned)

```
const obj1 = { name: 'x' }          // key is literally the text: name
```

```
const fieldName = 'email'
const obj2 = { [fieldName]: 'x' }   // key is the VALUE of fieldName -> { email: 'x' }
```

Final generic handler

```
const handleChange = (event) => {
  setFormData({
    ...formData,
    [event.target.name]: event.target.value,
  })
}
```

Step-by-step explanation

1. The user types in some input; React calls `handleChange` and passes the change event object.
2. `event.target` is that input element.
3. `event.target.name` is the field's name attribute (for example `phoneNumber`). Inside square brackets it becomes the key.
4. `event.target.value` is what the user has typed so far; it becomes the value of that key.
5. `...formData` keeps all other keys unchanged. `setFormData` stores the updated object.

Result: a single generic function reads the form data of different input fields. If the user types in the phone field, the key is `phoneNumber`; in the email field, the key is `email`.

All inputs use the same handler and have a name attribute

```
<input type="text" name="name" placeholder="Enter name"
       onChange={handleChange} />
<input type="text" name="phoneNumber" placeholder="Enter phone"
       onChange={handleChange} />
<input type="email" name="email" placeholder="Enter email"
       onChange={handleChange} />
<input type="password" name="password" placeholder="Enter password"
       onChange={handleChange} />
```

Common mistakes

1) Spelling of the `name` attribute differs from the state key (for example `phone` vs `phoneNumber`): a new unexpected key gets created.

2) Forgetting the square brackets: `{ event.target.name: ... }` is invalid; `{ [event.target.name]: ... }` is right.

3) Forgetting to add the `name` attribute on an input.

4) Forgetting `...formData` and wiping other fields.

Supplementary: controlled inputs

The lecture's approach uses `onChange` only. A fully 'controlled' input would also add `value={formData.name}` so React state is the single source of truth. This was not covered in the lecture; it is a natural next step, especially for clearing the form after submission.

14. Submitting the Form: onSubmit and preventDefault

14.1 Phase 1: first version of handleSubmit

When a form is submitted, the `onSubmit` event fires. Refer a function named `handleSubmit` on the form's `onSubmit`. Inside it, log the state, which contains the entire form data.

Initial handleSubmit (has a problem)

```
const handleSubmit = () => {
  console.log(formData)
}

// in the JSX
<form onSubmit={handleSubmit}>
```

Mechanism: the user types in the fields; our logic stores the data in the state; the user clicks the submit button; `onSubmit` fires; `handleSubmit` reads the state and logs the entire form data.

14.2 Problem

Test: enter name `Aman`, any phone number, email `rahul@mail.com` and a password, then click Submit. The class noticed that the **page reloaded**.

> **Instructor's warning**
> An unnecessary reload is not acceptable. If you reload the page unnecessarily in React or Angular, you will be called a bad developer.

The culprit is the **submit event**: the default behaviour of the submit event is to reload the browser window. This is not React's behaviour; it is the browser's default form behaviour. Thus we first have to prevent it.

14.3 Solution: event.preventDefault()

`handleSubmit` is a function called on an event, so React passes the event object again (this time the **submit event object**). On that object we call the `preventDefault` function. As its name says, it prevents the default behaviour of the event; for the submit event the default is to reload the browser window.

Correct handleSubmit

```
const handleSubmit = (event) => {
  event.preventDefault()    // stop the browser from reloading
  console.log(formData)     // read the form data from the STATE
}
```

Expected output: After typing the values and clicking Sign Up, the page does **not** reload and the console shows the entire form data as an object (JSON):

Console output (values depend on what you type)

```
{
  name: 'Aman',
  phoneNumber: '9876543210',
  email: 'aman@gmail.com',
  password: 'secret123'
}
```

This JSON object can be used as the **request body** of an API call (for example a register or create API) sent as a POST request so the server saves the data. The instructor said this will be discussed in the next episode.

14.4 Complete RegistrationForm.jsx

Final RegistrationForm.jsx (all three steps combined)

```
import { useState } from 'react'

export default function RegistrationForm() {
  const [formData, setFormData] = useState({
    name: '',
    phoneNumber: '',
    email: '',
    password: '',
  })

  const handleChange = (event) => {
    setFormData({
      ...formData,
      [event.target.name]: event.target.value,
    })
  }

  const handleSubmit = (event) => {
    event.preventDefault()
    console.log(formData)
  }

  return (
    <div>
      <h3>Register Today</h3>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Name</label>
          <input type="text" name="name" placeholder="Enter name"
                 onChange={handleChange} />
        </div>
        <br />
        <div>
          <label>Phone</label>
          <input type="text" name="phoneNumber" placeholder="Enter phone"
                 onChange={handleChange} />
        </div>
        <br />
        <div>
          <label>Email</label>
          <input type="email" name="email" placeholder="Enter email"
                 onChange={handleChange} />
        </div>
        <br />
        <div>
          <label>Password</label>
          <input type="password" name="password" placeholder="Enter password"
                 onChange={handleChange} />
        </div>
        <br />
        <button type="submit">Sign Up</button>
      </form>
    </div>
  )
}
```

Complete execution flow

1. Component renders; `formData` starts with four empty strings (Step 1).
2. User types in a field -> `onChange` fires -> React calls `handleChange` with the change event.
3. `handleChange` copies the old state, then sets the key named by `event.target.name` to `event.target.value` (Step 2).
4. User clicks Sign Up -> `onSubmit` fires -> `handleSubmit` runs with the submit event.

5. `preventDefault()` stops the page reload; `console.log(formData)` shows the data read from the state (Step 3).

> **Supplementary: the final App.jsx**
> For reference, the full routing file after all additions looks like this:

```
// src/App.jsx - final
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import NavigationBar from './components/NavigationBar'
import Home from './pages/Home'
import AboutUs from './pages/AboutUs'
import ContactUs from './pages/ContactUs'
import UserDetails from './pages/UserDetails'
import RegistrationForm from './pages/RegistrationForm'

function App() {
  return (
    <BrowserRouter>
      <NavigationBar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<AboutUs />} />
        <Route path="/contact" element={<ContactUs />} />
        <Route path="/user-details" element={<UserDetails />} />
        <Route path="/register" element={<RegistrationForm />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App

// NavigationBar.jsx - final (Register link added)
import { Link } from 'react-router-dom'

function NavigationBar() {
  return (
    <div>
      <Link to="/">Home</Link> |{' '}
      <Link to="/about">About Us</Link> |{' '}
      <Link to="/contact">Contact Us</Link> |{' '}
      <Link to="/register">Register</Link>
    </div>
  )
}

export default NavigationBar
```

15. End-of-Class Questions and Answers

The session ended with student questions. These are included because they answer common doubts.

15.1 Payment integration

Question: Will we learn payment integration and verification of payments on a website? **Answer:** Payment integration is a totally different thing. Payment gateways (PayPal and many other third-party vendors) provide APIs that you integrate. It is not part of this course. What the course covers: receive the form data on the front end and submit it to the back end, where an API exists (for example the admin registration API or the product save API), and then connect that API with the front end.

15.2 Generating PDF files (for example an inventory or invoice)

Question: If a client wants the inventory data as a PDF, how is it done? **Answer:** It is not included in the course, but it can be done. On the back end there are libraries to generate files and documents (PDF, Excel, doc). They provide functions to set data or write content into a PDF, similar to file handling functions. You call those functions, and the **response is the file, not JSON**. For example, for an invoice you call the library functions to build the entire PDF and send the PDF as the response. Search: 'how to send a file in the response' and 'how to create PDF files in Node Express'. A library mentioned is **PDFKit**: you create a document, set the header, font size and the content to write.

A student remarked that there will be many libraries and we must explore them through practice. The instructor agreed: the course duration is limited, so the focus is the basics of linking front end and back end; there will not be enough time to implement everything like full-fledged file handling.

15.3 Preventing repeated API calls while typing: Debouncing and Throttling

A student could not recall the name of a technique for avoiding repeated storing/processing of input as the user types. The instructor named it **debouncing**.

> **Debouncing (instructor's explanation)**
> After the user types, wait for a short time (for example around 250 ms or 100 ms depending on your application's bandwidth) before reading the value. This matters when typing triggers a request to the back end: sending a request for every single character would increase the server bill and hurt application performance. If the user wants to type 'iPhone', why send the request on 'i'? Let them type 'iph', wait briefly, then send the request.

The instructor also said there is another process called **throttling** and told the students to look up where it is used. The two things are **debouncing and throttling**.

> **Supplementary: debounce vs throttle (not explained in the lecture)**
> Debounce waits until the user has stopped for a given time and then runs once. Throttle runs at most once in every fixed time interval, no matter how often the event fires (for example while scrolling).

Debounce example

```
// Simple debounce helper (supplementary example)
function debounce(fn, delay) {
  let timer
  return (...args) => {
    clearTimeout(timer)                    // cancel the previous waiting call
    timer = setTimeout(() => fn(...args), delay)
  }
}
```

```
const search = debounce((text) => console.log('API call for', text), 300)
search('i')
search('ip')
search('iph')     // only this last call runs, 300 ms after the last keystroke
```

16. Revision Material

16.1 Concise summary of all major topics

- **Routing** = opening a component on a specific route (URL). It makes a single page application look like many pages.
- React has no built-in routing; install **react-router-dom** (`npm i react-router-dom`).
- Structure: `BrowserRouter` > `Routes` > `Route path=... element=...`.
- The navbar (anchor/Link) only changes the URL; React Router chooses the component.
- `path` in a route must exactly match the URL used in the link.
- The anchor tag reloads the browser; use **Link** (`to` prop) to navigate without a reload.
- **Programmatic routing**: call the `useNavigate` hook, which returns the `navigate` function; `navigate('/url')`.
- Forms: create one state object for the whole form (Step 1), capture values on `onChange` (Step 2), read from the state on submit (Step 3).
- React automatically passes the **event object** to any function called on an event. `event.target` = input element; `event.target.value` = typed value; `event.target.name` = name attribute.
- `setFormData` replaces the old state: use the **spread operator** to keep other keys.
- Use one generic `handleChange` with `[event.target.name]: event.target.value` and a `name` attribute on each input.
- `onSubmit` reloads the page by default: call `event.preventDefault()`. The form data JSON can be sent to an API (next lecture).
- Debouncing waits after typing before acting; throttling is a related technique.

16.2 Important definitions and syntax

Term / Syntax	Definition or explanation	Purpose
Routing	Opening a component on a specific route/URL.	Multiple screens in one SPA.
SPA	Single Page Application: one page whose component changes without reloading.	Fast, app-like navigation.
`BrowserRouter`	Wrapper component enabling routing in the browser.	Wrap the whole app.
`Routes`	Parent container of all `Route` components.	Mandatory wrapper for routes.
`Route`	`<Route path="/about" element={<AboutUs />} />`	URL to component mapping.
`Link`	`<Link to="/about">About</Link>`	Navigate without reload.
`useNavigate`	Hook returning a function to navigate programmatically.	Redirect from code.
`navigate('/x')`	Function (from `useNavigate`) that changes the URL.	Programmatic redirect.
Programmatic routing	Redirecting a user through code instead of a link click.	Login success etc.
`useState`	`const [formData, setFormData] = useState({...})`	Holds form data.
`onChange`	Event fired whenever an input's value changes.	Capture typed value.
Event object	Argument automatically passed by React to an event handler.	Info about the event.

Term / Syntax	Definition or explanation	Purpose
`event.target`	The element on which the event occurred.	Replaces getElementById.
`event.target.value`	Current value of the input field.	Read typed data.
`event.target.name`	The `name` attribute of the input.	Becomes the state key.
Spread `...`	`{ ...formData, name: v }`	Copy old keys, change one.
`[key]: value`	Computed property name in an object.	Dynamic key.
`onSubmit`	Event fired when the form is submitted.	Handle the submission.
`event.preventDefault()`	Stops the default behaviour of the event.	Stop page reload on submit.

16.3 Key differences between related concepts

Comparison	Concept A	Concept B
Library vs framework	Library (React): small, add what you need.	Framework: many features built in.
Anchor vs Link	`<a href>`: reloads the page.	`<Link to>`: no reload.
`href` vs `to`	`href` is for anchor tags.	`to` is for the `Link` component.
Link vs navigate	User clicks a link.	Code decides the redirect.
Navbar vs Router	Navbar changes the URL.	Router opens the matching component.
Many states vs one state	Many states: hard to maintain.	One object: preferred.
onChange vs onSubmit	Fires while typing: store data in state.	Fires on submit: read state, send data.
getElementById vs event.target	Core JS way to get the element.	React way: element comes from the event.
id vs name for forms	id: works but not preferred for forms.	name: preferred; used as the state key.
Reference vs call	`onClick={handler}`: React calls it later.	`onClick={handler()}`: runs immediately (wrong).

16.4 Common mistakes to avoid

Mistake	Why it is wrong	Correction
Using `<a>` for navigation in React	Browser reloads.	Use `Link`.
`href` on `Link`	Not supported.	Use `to`.
Route not inside `Routes`	Routing will not work properly.	Wrap routes in `Routes`.
App not inside `BrowserRouter`	Routing is not enabled.	Wrap the app.
`path` differs from link URL	No match, so no component opens.	Keep both exactly equal.
Route not registered for a programmatic URL	URL changes but nothing opens.	Add a `Route` for every URL.
`setFormData({ name: v })`	Replaces the whole state.	`{ ...formData, name: v }`
Hard coded key in generic handler	All data goes to one key.	`[event.target.name]`
Missing `name` attribute	`event.target.name` is empty.	Add `name` to every input.
No `preventDefault` in submit	Page reloads and state is lost.	`event.preventDefault()`

Mistake	Why it is wrong	Correction
Reading input fields at submit time	Defeats the purpose of the state.	Read from the state.

16.5 Important interview questions and answers

Questions 1-12 are based on lecture content. Questions marked (extra) go beyond the lecture.

Question	Answer
1. What is routing?	Opening a component on a specific route (URL). It lets a single page application show different screens.
2. Does React have built-in routing?	No. React is a small library; use the third-party package react-router-dom.
3. Name the three main components of react-router-dom used in the lecture.	`BrowserRouter`, `Routes`, `Route`.
4. What is the role of `Route`?	It defines the mapping from a URL (`path`) to the component to open (`element`).
5. Why is `Routes` needed?	It is the container (parent) in which all `Route` components must be wrapped.
6. Why wrap the app in `BrowserRouter`?	To enable routing in the browser for the whole application.
7. Why not use an anchor tag for navigation?	It reloads the browser. A single page application must not reload.
8. Difference between `<a>` and `Link`?	Both send the user to a URL; `Link` uses `to` and does not reload.
9. What is programmatic routing and how is it done?	Redirecting through code, e.g. after login. Call `useNavigate()` to get `navigate`, then `navigate('/path')`.
10. Why keep form data in a single state object?	Easier to maintain, less complexity, fewer unnecessary re-renders than many separate states.
11. What is `event.target`?	The HTML element on which the event occurred. Its `.value` is the typed data.
12. Why use `preventDefault()` in `handleSubmit`?	The default submit behaviour reloads the page; preventing it keeps the SPA and the state intact.
13. Why use the spread operator when updating form state?	`setFormData` replaces the state; the spread keeps the other keys.
14. How does one `handleChange` serve many inputs?	Each input has a `name` equal to its state key; the handler uses `[event.target.name]: event.target.value`.
15. What is debouncing?	Waiting a short time after the user stops typing before reading the value or calling the API, to avoid too many requests.
16. (extra) What does a controlled input mean?	An input whose value is supplied by React state via the `value` prop and updated via `onChange`.
17. (extra) Where must hooks like `useNavigate` be called?	At the top level of a function component (or custom hook), not inside nested functions or conditions.

MCQ pointers

Remember: component names `Link`, `Route`, `Routes`, `BrowserRouter` are capitalised; `Link` uses `to`; `useNavigate` returns `navigate`; `event.target.value` gives the typed text; the default behaviour of submit is a page reload; `preventDefault` stops it.

16.6 Practice exercises (solutions not included)

Easy

1. Set up a new Vite React project, install `react-router-dom`, and create Home and About pages with routes `/` and `/about`.
2. Add a navigation bar using `Link`. Observe in the browser whether the page reloads, then replace `Link` with `<a href>` and observe the difference.
3. Create a Services page at `/services` and add its link.
4. In a component, log the click event object of a button and explore the `target` property in the console.

Medium

1. Create a Login page with a button that, when clicked, redirects to a Dashboard page using `useNavigate`.
2. Build a registration form with name, email and password using one state object, one `handleChange` and the `name` attribute. Log the state on submit.
3. Intentionally remove `...formData` from your handler and describe what goes wrong. Then fix it.
4. Intentionally remove `preventDefault()` and describe what happens to your form data.

Challenging

1. Extend the form with a date of birth field and a city field without adding any new handler function.
2. On submit, check whether any state value is empty and show a message (the lecture said an empty state means the user has not entered data). Only log the data if all fields are filled.
3. After a successful submit, use `navigate` to send the user to a 'Thank You' page.
4. Research debouncing and add a debounced search box that logs the text 300 ms after the user stops typing. Then research throttling and describe when you would use it instead.

16.7 Final revision checklist

[ ] I can explain what routing is and why a single page application needs it.

[ ] I know that React has no built-in routing and why react-router-dom is used.

[ ] I can write BrowserRouter, Routes and Route from memory (with path and element).

[ ] I know the navbar only changes the URL, and the Router opens the matching component.

[ ] I know path and the link's URL must match exactly.

[ ] I can explain why anchor tags cause reload and how Link solves it (the `to` prop).

[ ] I can explain programmatic routing and write useNavigate and navigate('/url').

[ ] I can create a form UI and the single state object for its data.

[ ] I can explain the three steps: create state, capture on change, read state on submit.

[ ] I know React passes the event object automatically, and I can use event.target and event.target.value.

[ ] I understand why setFormData needs the spread operator.

[ ] I can write one generic handleChange using the name attribute and [event.target.name].

[ ] I know why onSubmit needs preventDefault and what the console output should look like.

[ ] I know the idea of debouncing and that throttling exists.

[ ] I have completed at least the easy and medium practice exercises.